# Minimum-Cable Campus Networks: A Greedy Minimum Spanning Tree Approach

Sridhar Kumar
*Department of Computer Science*
*University of Florida*
Gainesville, FL, USA
f.sridharkumar@ufl.edu

Daen P Khan
*Department of Computer Science*
*University of Florida*
Gainesville, FL, USA
dpatan@ufl.edu

***Abstract*—Campus telecommunications planning requires a connected backbone between buildings, but candidate routes differ in cable length. We model this planning problem as a weighted undirected graph and seek a connected set of routes with minimum total length. Kruskal's greedy algorithm sorts routes by length and accepts a route only when it joins two currently disconnected components. A cut-property exchange argument proves that every accepted route is safe and that the final network is a minimum spanning tree. With a disjoint-set data structure, the implementation runs in $O(E \log E)$ time and uses $O(E+V)$ space. Tests and a reproducible experiment over 28 connected synthetic graph families support the predicted growth.**

***Index Terms*—greedy algorithms, minimum spanning tree, Kruskal's algorithm, campus cabling, graph optimization**

## I. Introduction

Campus network planners must connect buildings through feasible pathways such as conduit, utility tunnels, and approved interbuilding backbone routes. Each possible route has a measured cable length and installation plan. The base planning question is to connect every building while minimizing the total amount of cable installed. Campus infrastructure standards describe these pathways as a distinct engineering concern [1].

This paper studies the base minimum-cable problem as a greedy algorithm application. The buildings and feasible routes form a weighted graph, and the desired network is a minimum spanning tree (MST). Kruskal's algorithm makes a local choice: among the remaining routes, choose the shortest route that does not create a cycle. The cut property shows why this local rule preserves a globally optimal solution.

Our contributions are:

- a graph abstraction of minimum-cable campus backbone planning;
- a formal Kruskal algorithm using a disjoint-set forest;
- a cut-property exchange proof of optimality;
- an $O(E \log E)$ running-time analysis; and
- an implementation, correctness test suite, and timing experiment.

## II. Related Work and Domain Context

Minimum spanning trees are used when a set of weighted links must connect all vertices at minimum total weight. Kruskal's algorithm is a standard greedy solution and uses disjoint sets to test whether a candidate edge joins different components [2]. The engineering motivation here comes from campus outside-plant telecommunications. Real projects can also require redundancy, route diversity, capacity limits, and construction costs [1]. Those requirements are outside the base optimization model so that the greedy structure can be analyzed directly.

## III. Problem Formulation

### A. Real-World Problem

A campus has a collection of buildings and a survey of permitted pathways between pairs of buildings. For each pathway, the survey records the cable length needed to connect its endpoints. Every building must be able to reach every other building through the installed network, possibly through intermediate buildings, while the total installed cable is minimized.

The model assumes that every supplied route is physically feasible, lengths already reflect the selected pathway, and a building may forward traffic. It does not include backup links, link capacity, excavation cost, or a requirement for two physically diverse paths. Those constraints can lead to richer network design problems, but they are not part of this report's objective.

### B. Mathematical Abstraction

Let $G = (V, E)$ be a connected undirected graph. Each vertex $v \in V$ is a building. Each edge $e = \{u, v\} \in E$ is an approved route, with positive weight $w(e)$ equal to its required cable length. We seek a subset $T \subseteq E$ such that $(V, T)$ is connected and

$$W(T) = \sum_{e \in T} w(e) \tag{1}$$

is minimum. Any connected solution containing a cycle can remove one cycle edge and remain connected. Since all weights are positive, an optimal solution is acyclic and therefore contains exactly $|V| - 1$ routes. The problem is thus the minimum spanning tree problem.

**Problem (Minimum-Cable Network).** *Input:* a connected weighted graph $G = (V, E)$.
*Output:* a spanning tree $T$ minimizing $W(T)$.

**Algorithm 1** Kruskal minimum-cable network

**Require:** connected graph $G = (V, E)$ with positive route lengths $w$
**Ensure:** minimum-length set of routes connecting all buildings

```
T ← ∅
for all v ∈ V do
    MAKESET(v)
end for
sort E by nondecreasing w(e)
for all e = {u, v} ∈ E in sorted order do
    if FIND(u) ≠ FIND(v) then
        T ← T ∪ {e}
        UNION(u, v)
        if |T| = |V| − 1 then
            return T
        end if
    end if
end for
report that the route graph is disconnected
```

*Cost:* total route length, with running time measured separately.

## IV. Greedy Algorithm

### A. Strategy

Kruskal's algorithm sorts all routes in nondecreasing order of length. It starts with each building as its own component. A route is installed when its endpoints are in different components; the two components are then merged. A route whose endpoints are already connected would create a cycle and is skipped. The algorithm stops after $|V| - 1$ routes have been accepted.

In domain terms, the planning team considers the shortest available cable route first. It installs that route when it connects two groups of buildings that cannot yet communicate. Routes internal to one existing group are not needed for the minimum-cable backbone because they only create cycles.

### B. Worked Example

Suppose four buildings have feasible routes of lengths 1, 2, 3, 4, and 5 meters, where the first three selected routes connect all four buildings. Kruskal accepts the 1-meter route, then the 2-meter route, and then the 3-meter route. If the 4-meter route would join buildings already connected by those choices, it is skipped. The resulting tree has total length $1 + 2 + 3 = 6$ meters and connects every building with no cycle.

### C. Implementation Notes

The C++17 implementation stores each route as its two endpoints, a signed 64-bit length, and its input order. It validates positive lengths and endpoint ranges before sorting. Ties are deterministic: route length, normalized endpoints, and input order are used as tie breakers. Path compression and union by rank implement the disjoint-set forest. The implementation detects disconnected input and checks for signed 64-bit overflow when accumulating the total length.

## V. Complexity Analysis

Sorting $E$ routes costs $O(E \log E)$. With path compression and union by rank, all disjoint-set operations over the route scan cost $O(E\alpha(V))$, where $\alpha$ is the inverse Ackermann function [2]. Therefore,

$$T(V, E) = O(E \log E). \tag{2}$$

The route list, selected tree, and disjoint-set arrays use $O(E + V)$ space. If every pair of buildings is a candidate route, $E = \Theta(V^2)$ and the bound becomes $O(V^2 \log V)$.

## VI. Proof of Correctness

*Lemma 1 (Safe greedy route):* Let $A$ be the set of routes already selected by Kruskal, and suppose some MST $M$ contains $A$. If the next accepted route $e$ joins two different components of $(V, A)$, then some MST contains $A \cup \{e\}$.

*Proof:* Let $C$ be the component of $(V, A)$ containing one endpoint of $e$. The other endpoint lies outside $C$, so $e$ crosses the cut $(C, V \setminus C)$. At the time $e$ is selected, it is a lightest eligible route crossing this cut: a lighter route crossing the cut would also join different components and would have appeared earlier in the sorted scan.

If $e \in M$, the claim is immediate. Otherwise, adding $e$ to the tree $M$ creates one cycle. That cycle contains an edge $f$ crossing the cut. No edge of $A$ crosses between two components of $(V, A)$, so $f \notin A$. Since $e$ is a lightest eligible crossing route, $w(e) \le w(f)$. Replacing $f$ by $e$ gives $M' = M - \{f\} + \{e\}$, a spanning tree with no greater weight. The minimality of $M$ implies that $M'$ is also an MST, and it contains $A \cup \{e\}$. ∎

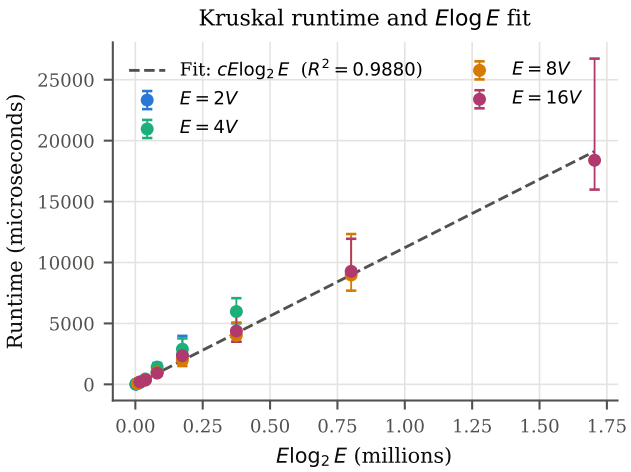
*Theorem 1:* Algorithm 1 returns a minimum-length connected cable network for every connected input graph.

*Proof:* Initially $A = \emptyset$, which is contained in an MST. By Lemma 1, after each accepted route there remains an MST containing all routes selected so far. Because the input graph is connected, the algorithm eventually accepts $|V| - 1$ routes. The accepted routes are acyclic because a route is accepted only when it joins distinct components, so they form a spanning tree. An MST contains all of this tree's $|V| - 1$ routes and has the same number of edges; therefore the selected tree itself is an MST and has minimum total cable length. ∎

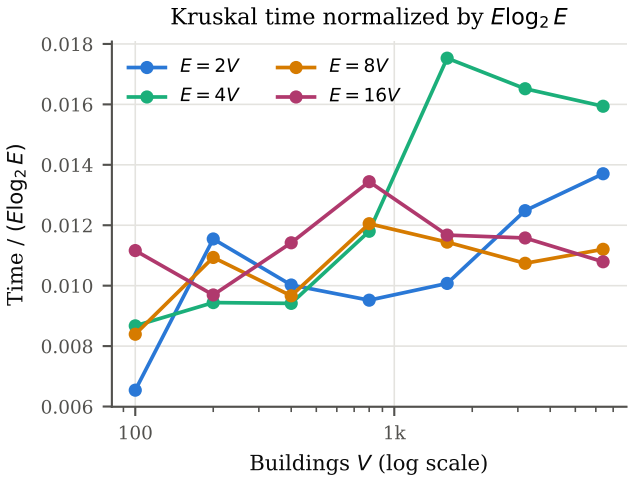
## VII. Experimental Validation

### A. Correctness Testing

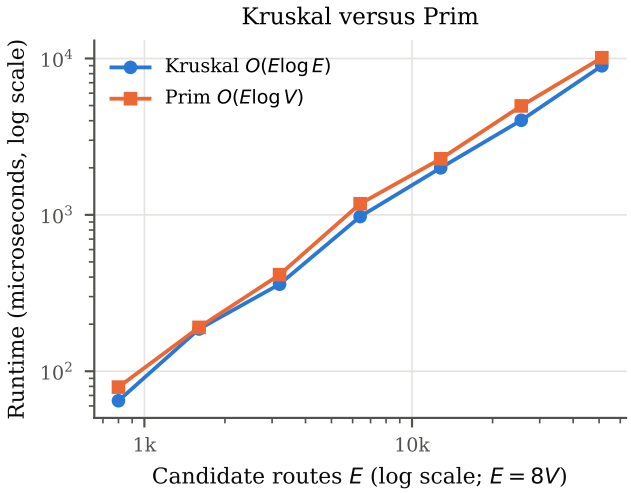
The test suite covers the six-route example, malformed input, disconnected graphs, parallel routes, equal weights, a single building, large integer weights, and 64-bit overflow. Small graphs are compared with exhaustive enumeration of all candidate spanning trees. A seeded 800-building graph with 16,000 routes is checked against an independent Prim implementation. A 5,000-building sparse graph with a known optimum provides an additional stress case. All eleven test groups are intended to check both the CLI output and the selected tree structure.



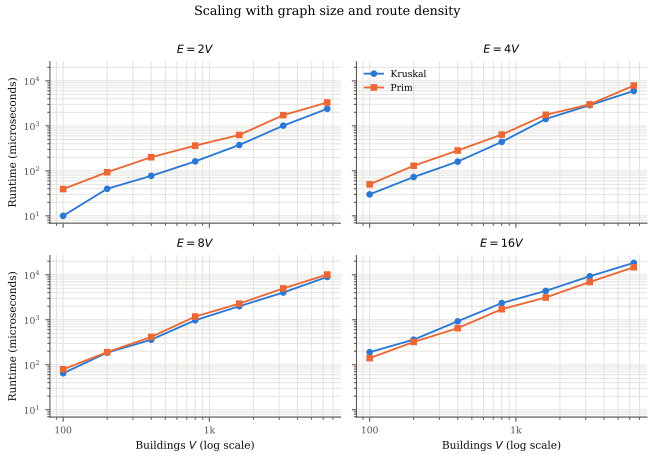

(a) Runtime and $E \log E$ fit.




(b) Runtime normalized by $E \log E$.




(c) Kruskal versus Prim on $E = 8V$.




(d) Scaling across route densities.

Fig. 1. Experimental results for Kruskal's minimum-cable algorithm on 28 connected synthetic graph families.

### B. Synthetic Graphs and Setup

The benchmark generates connected random graphs with $V \in \{100, 200, 400, 800, 1600, 3200, 6400\}$ and $E \in \{2V, 4V, 8V, 16V\}$. A random spanning chain is inserted first, so every generated graph is connected; remaining routes are sampled without duplicate undirected endpoints. Route lengths are positive random integers. Graph generation occurs outside the timer. Each of the 28 graph families uses a fixed seed and seven trials. Short runs are repeated inside each trial, and the reported time is the average duration of one algorithm call. The same graphs are timed with an independent Prim implementation, producing 392 algorithm observations in the CSV (196 per algorithm).

The experiment was compiled with MinGW GCC 6.3.0 using `-std=c++17 -O2 -Wall -Wextra -pedantic` on Windows. Processor and memory details should be recorded when the final submission run is performed.

### C. Results

The smallest Kruskal case, $V = 100$ and $E = 200$, had a median runtime of $10.00\,\mu$s. At the largest graph, $V = 6400$ and $E = 102400$, the median Kruskal time was $18391.33\,\mu$s and the median Prim time was $14680.33\,\mu$s. A line proportional to $E \log_2 E$ fitted through the origin gave $R^2 = 0.98804$ across the 28 Kruskal median points (Fig. 1a). The normalized values in Fig. 1b remain comparable across route densities. Prim was between 1.02 and 1.24 times the Kruskal time at $E = 8V$, with the ratio varying by graph size (Fig. 1c). The density panels in Fig. 1d show that both methods slow as candidate routes become denser. These measurements agree with the derived $O(E \log E)$ upper bound; they do not replace the correctness proof or establish an asymptotic bound by themselves.

TABLE I
MEDIAN KRUSKAL RUNTIME IN MICROSECONDS

| $V$ | $E = 2V$ | $E = 4V$ | $E = 8V$ | $E = 16V$ |
|---|---|---|---|---|
| 100 | 10.00 | 29.98 | 64.75 | 190.09 |
| 200 | 39.92 | 72.83 | 186.21 | 361.14 |
| 400 | 77.32 | 160.32 | 360.08 | 924.04 |
| 800 | 162.12 | 439.68 | 975.16 | 2347.36 |
| 1600 | 375.38 | 1418.50 | 1998.00 | 4376.38 |
| 3200 | 1010.12 | 2884.75 | 4026.50 | 9274.25 |
| 6400 | 2393.33 | 5975.00 | 8973.67 | 18391.33 |

TABLE II
MEDIAN PRIM RUNTIME IN MICROSECONDS

| $V$ | $E = 2V$ | $E = 4V$ | $E = 8V$ | $E = 16V$ |
|---|---|---|---|---|
| 100 | 39.36 | 50.06 | 79.17 | 140.41 |
| 200 | 93.26 | 129.98 | 190.77 | 320.66 |
| 400 | 200.60 | 283.08 | 414.52 | 650.08 |
| 800 | 363.56 | 639.68 | 1176.96 | 1717.56 |
| 1600 | 626.62 | 1758.50 | 2282.88 | 3125.12 |
| 3200 | 1729.50 | 3000.12 | 4972.62 | 6907.50 |
| 6400 | 3307.33 | 7845.00 | 10111.33 | 14680.33 |

## VIII. DISCUSSION AND LIMITATIONS

Kruskal's method is appropriate when the planning objective is the sum of positive route lengths and every building only needs to be connected. Its greedy decisions are irrevocable, but the cut-property proof guarantees that each decision can be extended to an optimal tree. The disjoint-set data structure keeps cycle checks efficient even when the route list is large.

The model does not guarantee network resilience. A real campus backbone may need two independent paths, capacity guarantees, existing-duct constraints, construction costs, or protection against a single route failure. Those goals require a more expressive network design model. The experiment also uses synthetic graphs and measures algorithm time rather than construction time; the results validate implementation growth, not real project cost.

## IX. CONCLUSION

We formulated minimum-cable campus backbone planning as a weighted graph problem and solved it with Kruskal's greedy algorithm. The cut-property exchange argument proves that the selected routes form an MST. The C++17 implementation runs in $O(E \log E)$ time, handles invalid and disconnected inputs, and was tested against exhaustive and Prim-based references. A fixed seed experiment over 28 connected graph families produced timing growth consistent with $E \log E$.

## REFERENCES


[1] Yale University, "IT infrastructure design guidelines and performance standards," sec. 1.16, Interbuilding Backbone Pathways, 2025. [Online]. Available: Yale IT infrastructure design guidelines.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.


## APPENDIX A
## RUNTIME VALIDATION CODE

The following source files generate the connected test graphs, run Kruskal's algorithm and the Prim baseline, measure repeated trials, and write the CSV data used for the experimental figures. The implementation files included below are the algorithm code called by the benchmark.

Listing 1. Greedy benchmark harness

```
#include "../src/mst.hpp"

#include <algorithm>
#include <chrono>
#include <cstdint>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <functional>
#include <random>
#include <stdexcept>
#include <unordered_set>
#include <vector>
#include <queue>

namespace {

std::uint64_t route_key(int first, int second) {
    std::uint32_t first_id = static_cast<std::uint32_t>(first);
    std::uint32_t second_id = static_cast<std::uint32_t>(second);
    if (first_id > second_id) {
        std::swap(first_id, second_id);
    }
    return (static_cast<std::uint64_t>(first_id) << 32) | second_id;
}

std::vector<cabling::Edge> make_graph(int vertices, int edge_count,
                                      std::uint64_t seed) {
    std::mt19937_64 random(seed);
    std::uniform_int_distribution<int> vertex(0, vertices - 1);
    std::uniform_int_distribution<int> length(1, 1000000);
    std::unordered_set<std::uint64_t> used;
    used.reserve(static_cast<std::size_t>(edge_count * 2));
    std::vector<cabling::Edge> edges;
    edges.reserve(static_cast<std::size_t>(edge_count));

    // The chain guarantees that every generated graph is connected.
    for (int index = 1; index < vertices; ++index) {
        edges.push_back({index - 1, index, length(random), edges.size()});
        used.insert(route_key(index - 1, index));
    }
    while (static_cast<int>(edges.size()) < edge_count) {
        int first = vertex(random);
        int second = vertex(random);
        if (first == second || !used.insert(route_key(first, second)).second) {
            continue;
        }
        edges.push_back({first, second, length(random), edges.size()});
    }
    return edges;
}

int inner_repetitions(int vertices) {
    if (vertices <= 200) return 100;
    if (vertices <= 800) return 25;
    if (vertices <= 3200) return 8;
    return 3;
}

long long prim_total_length(int vertices,
                            const std::vector<cabling::Edge>& edges) {
    std::vector<std::vector<std::pair<int, int>>> adjacency(vertices);
    for (const cabling::Edge& edge : edges) {
        adjacency[edge.from].push_back({edge.length, edge.to});
        adjacency[edge.to].push_back({edge.length, edge.from});
    }

    using Candidate = std::pair<int, int>;
    std::priority_queue<Candidate, std::vector<Candidate>, std::greater<Candidate>>
        frontier;
    std::vector<bool> visited(vertices, false);
    frontier.push({0, 0});
    long long total = 0;
    int reached = 0;
    while (!frontier.empty() && reached < vertices) {
        const Candidate current = frontier.top();
        frontier.pop();
        const int length = current.first;
        const int vertex = current.second;
        if (visited[vertex]) {
            continue;
        }
        visited[vertex] = true;
        total += length;
        ++reached;
        for (const std::pair<int, int>& next : adjacency[vertex]) {
            if (!visited[next.second]) {
                frontier.push({next.first, next.second});
            }
        }
    }
    if (reached != vertices) {
        throw std::runtime_error("benchmark graph unexpectedly disconnected");
    }
    return total;
}

}  // namespace

int main(int argc, char** argv) {
    if (argc > 2) {
        std::cerr << "usage: benchmark [output.csv]\n";
```

```
        return 1;
    }
    std::ofstream file;
    std::ostream* output = &std::cout;
    if (argc == 2) {
        file.open(argv[1]);
        if (!file) {
            std::cerr << "cannot␣open␣benchmark␣output␣file\n";
            return 1;
        }
        output = &file;
    }
    const std::vector<int> vertex_counts{100, 200, 400, 800, 1600, 3200, 6400};
    const std::vector<int> density_factors{2, 4, 8, 16};
    constexpr int trials = 7;
    std::uint64_t checksum = 0;

    *output << "algorithm,vertices,edges,density_factor,seed,trial,elapsed_us,
       total_length\n";
    *output << std::fixed << std::setprecision(3);
    for (int vertices : vertex_counts) {
        for (int factor : density_factors) {
            const int edge_count = factor * vertices;
            const std::uint64_t seed = 20261024ULL +
                static_cast<std::uint64_t>(vertices * 100 + factor);
            const std::vector<cabling::Edge> edges =
                make_graph(vertices, edge_count, seed);
            const int repetitions = inner_repetitions(vertices);

            for (int trial = 0; trial < trials; ++trial) {
                const auto kruskal_start = std::chrono::steady_clock::now();
                long long kruskal_length = 0;
                for (int repeat = 0; repeat < repetitions; ++repeat) {
                    const cabling::MstResult result =
                        cabling::kruskal(vertices, edges);
                    kruskal_length = result.total_length;
                    checksum ^= static_cast<std::uint64_t>(kruskal_length + repeat);
                }
                const auto kruskal_finish = std::chrono::steady_clock::now();
                const double kruskal_us =
                    std::chrono::duration<double, std::micro>(kruskal_finish -
    kruskal_start).count()
                    / repetitions;
                *output << "Kruskal," << vertices << ',' << edge_count << ',' <<
    factor << ','
                        << seed << ',' << trial << ',' << kruskal_us << ','
                        << kruskal_length << '\n';

                const auto prim_start = std::chrono::steady_clock::now();
                long long prim_length = 0;
                for (int repeat = 0; repeat < repetitions; ++repeat) {
                    prim_length = prim_total_length(vertices, edges);
                    checksum ^= static_cast<std::uint64_t>(prim_length + repeat +
    17);
                }
                const auto prim_finish = std::chrono::steady_clock::now();
                const double prim_us =
                    std::chrono::duration<double, std::micro>(prim_finish -
    prim_start).count()
                    / repetitions;
                if (prim_length != kruskal_length) {
                    throw std::runtime_error("Kruskal␣and␣Prim␣totals␣disagree");
                }
                *output << "Prim," << vertices << ',' << edge_count << ',' << factor
     << ','
                        << seed << ',' << trial << ',' << prim_us << ','
                        << prim_length << '\n';
            }
        }
    }
    std::cerr << "Benchmark␣checksum:␣" << checksum << '\n';
}
```

Listing 2. Minimum-spanning-tree interface

```
#ifndef GREEDY_MST_HPP
#define GREEDY_MST_HPP

#include <cstddef>
#include <vector>

namespace cabling {

struct Edge {
    int from;
    int to;
    long long length;
    std::size_t input_order;
};

struct MstResult {
    long long total_length;
    std::vector<Edge> routes;
};

// Throws std::invalid_argument for an invalid or disconnected graph and
// std::overflow_error if the total length exceeds a signed 64-bit integer.
MstResult kruskal(int building_count, std::vector<Edge> routes);

}  // namespace cabling

#endif
```

Listing 3. Minimum-spanning-tree implementation

```
#include "mst.hpp"

#include <algorithm>
#include <limits>
#include <numeric>
#include <stdexcept>
#include <utility>
#include <vector>

namespace cabling {
namespace {

class DisjointSet {
public:
    explicit DisjointSet(int size) : parent_(size), rank_(size, 0) {
        std::iota(parent_.begin(), parent_.end(), 0);
    }

    int find(int vertex) {
        if (parent_[vertex] != vertex) {
            parent_[vertex] = find(parent_[vertex]);
        }
        return parent_[vertex];
    }

    bool unite(int first, int second) {
        first = find(first);
        second = find(second);
        if (first == second) {
            return false;
        }
        if (rank_[first] < rank_[second]) {
            std::swap(first, second);
        }
        parent_[second] = first;
        if (rank_[first] == rank_[second]) {
            ++rank_[first];
        }
        return true;
    }

private:
    std::vector<int> parent_;
    std::vector<int> rank_;
};

bool shorter_first(const Edge& first, const Edge& second) {
    if (first.length != second.length) {
        return first.length < second.length;
    }
    const auto first_ends = std::minmax(first.from, first.to);
    const auto second_ends = std::minmax(second.from, second.to);
    if (first_ends != second_ends) {
        return first_ends < second_ends;
    }
    return first.input_order < second.input_order;
}

}  // namespace

MstResult kruskal(int building_count, std::vector<Edge> routes) {
    if (building_count <= 0) {
        throw std::invalid_argument("building␣count␣must␣be␣positive");
    }
    for (const Edge& route : routes) {
        if (route.from < 0 || route.to < 0 || route.from >= building_count ||
            route.to >= building_count || route.from == route.to ||
            route.length <= 0) {
            throw std::invalid_argument("route␣has␣invalid␣endpoints␣or␣length");
        }
    }

    std::sort(routes.begin(), routes.end(), shorter_first);
    DisjointSet components(building_count);
    MstResult result{0, {}};
    result.routes.reserve(static_cast<std::size_t>(building_count - 1));
    if (building_count == 1) {
        return result;
    }

    for (const Edge& route : routes) {
        if (!components.unite(route.from, route.to)) {
            continue;
        }
        if (result.total_length >
            std::numeric_limits<long long>::max() - route.length) {
            throw std::overflow_error("total␣cable␣length␣exceeds␣64-bit␣limit");
        }
        result.total_length += route.length;
        result.routes.push_back(route);
        if (result.routes.size() == static_cast<std::size_t>(building_count - 1)) {
            return result;
        }
    }

    throw std::invalid_argument("permitted␣routes␣do␣not␣connect␣all␣buildings");
}

}  // namespace cabling
```

## CODE AVAILABILITY

The implementation, tests, benchmark, raw CSV results, plotting script, and generated figures are available in the repository under `greedy/`. The complete code repository is available at github.com/Sridhar-97k/AOA_Project1. From

the repository root, compile the report with `latexmk -pdf greedy/greedy_report.tex`. Build and experiment commands are documented in `greedy/README.md`.